PR-9

Node.js Authentication and Authorization using HTTP Module

1. Introduction

This project demonstrates **Basic Authentication and Authorization** using the built-in Node.js `http` module.

The application has two users:

Username	Password	Role
`admin`	`1234`	`admin`
`student`	`1234`	`user`

The server checks the username and password sent through HTTP request headers.

After authentication, it checks the user's role when accessing the `/admin` route.

---

2. Authentication vs Authorization

Authentication

Authentication answers:

> "Who are you?"

In this application, the username and password are checked against the users stored in the `users` array.

For example:

```
Username: admin
Password: 1234
```

If the credentials are correct, the user is authenticated.

If they are incorrect:

```
401 Unauthorized
```

is returned.

Authorization

Authorization answers:

> "What are you allowed to access?"

After authentication, the application checks the user's role.

For example:

```
admin → admin
student → user
```

Only users with the `admin` role can access:

```
/admin
```

If a normal user tries to access `/admin`, the server returns:

```
403 Forbidden
```

---

3. Project Structure

Create a folder:

```
node-auth-demo/
│
└── server.js
```

The complete code will be inside `server.js`.

---

4. Complete Code

Create a file named:

```
server.js
```

Add the following code:

```
const http = require("http");

const users = [
  {
    username: "admin",
    password: "1234",
    role: "admin"
  },
  {
    username: "student",
    password: "1234",
    role: "user"
  }
];
```

```
const server = http.createServer((req, res) => {

  // Get username and password from headers
  const username = req.headers.username;
  const password = req.headers.password;

  // ---------- AUTHENTICATION ----------
  const user = users.find(
    u => u.username === username && u.password === password
  );

  if (!user) {
    res.writeHead(401, {
      "Content-Type": "application/json"
    });

    return res.end(
      JSON.stringify({
        message: "Authentication failed"
      })
    );
  }

  // ---------- AUTHORIZATION ----------
  if (req.url === "/admin") {

    if (user.role !== "admin") {
      res.writeHead(403, {
        "Content-Type": "application/json"
      });

      return res.end(
        JSON.stringify({
          message: "Access denied. Admin only."
        })
      );
    }

    res.writeHead(200, {
      "Content-Type": "application/json"
    });

    return res.end(
      JSON.stringify({
        message: "Welcome Admin!",
        user: user.username
      })
```

```
    );
  }

  // Normal authenticated user
  res.writeHead(200, {
    "Content-Type": "application/json"
  });

  res.end(
    JSON.stringify({
      message: "Login successful",
      user: user.username,
      role: user.role
    })
  );
});

server.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

---

5. How the Code Works

Step 1: Import HTTP Module

```
const http = require("http");
```

Node.js provides the `http` module for creating HTTP servers.

No external package is required.

Step 2: Create Users

```
const users = [
  {
    username: "admin",
    password: "1234",
    role: "admin"
```

```
  },
  {
    username: "student",
    password: "1234",
    role: "user"
  }
];
```

This array acts as a temporary database.

Each user has:

- `username`
- `password`
- `role`

---

6. Creating the Server

```
const server = http.createServer((req, res) => {
```

The callback receives two objects:

`req`

Contains information about the incoming request.

For example:

```
req.url
req.method
req.headers
```

`res`

Used to send a response back to the client.

7. Reading Username and Password

The application reads the credentials from HTTP headers:

```
const username = req.headers.username;
const password = req.headers.password;
```

For example, the request can contain:

```
username: admin
password: 1234
```

8. Authentication

The following code searches for a matching user:

```
const user = users.find(
  u => u.username === username && u.password === password
);
```

If a matching user is found:

```
user
```

contains that user's information.

For example:

```
{
  username: "admin",
  password: "1234",
```

```
  role: "admin"
}
```

If no matching user is found, `user` becomes:

```
undefined
```

---

9. Authentication Failure

The application checks:

```
if (!user)
```

If the user does not exist, it sends:

```
401 Unauthorized
```

Response:

```
{
  "message": "Authentication failed"
}
```

---

10. Authorization

After authentication, the application checks whether the requested URL is:

```
/admin
```

```
if (req.url === "/admin")
```

It then checks the user's role:

```
if (user.role !== "admin")
```

If the user is not an admin, the server returns:

```
403 Forbidden
```

Response:

```
{
  "message": "Access denied. Admin only."
}
```

---

ll. Admin Access

If the authenticated user is an admin, the server returns:

```
{
  "message": "Welcome Admin!",
  "user": "admin"
}
```

with HTTP status:

```
200 OK
```

---

l2. Running the Project

Step l: Install Node.js

Make sure Node.js is installed.

Check:

```
node --version
```

You should see something similar to:

```
v22.x.x
```

Step 2: Open the Project Folder

Open a terminal inside:

```
node-auth-demo
```

Your folder should contain:

```
server.js
```

Step 3: Run the Server

Execute:

```
node server.js
```

You should see:

```
Server running on http://localhost:3000
```

The server is now running.

---

13. Testing Using Postman

Open **Postman**.

We will test three scenarios:

1. Admin accessing `/admin`
2. Normal user accessing `/admin`
3. Invalid credentials

---

14. Test 1 — Admin Access

Create a new request in Postman.

Method

```
GET
```

URL

```
http://localhost:3000/admin
```

Go to:

```
Headers
```

Add:

Key	Value
`username`	`admin`
`password`	`1234`

Your request should look like:

```
GET http://localhost:3000/admin

Headers:

username: admin
password: 1234
```

Click:

```
Send
```

Expected Response

Status:

```
200 OK
```

Response:

```
{
  "message": "Welcome Admin!",
  "user": "admin"
}
```

This means:

```
Authentication → Successful
Authorization → Successful
```

---

15. Test 2 — Normal User Accessing Admin Route

Create another request:

```
GET http://localhost:3000/admin
```

Headers:

Key	Value
`username`	`student`
`password`	`1234`

Click:

```
Send
```

Expected Response

Status:

```
403 Forbidden
```

Response:

```
{
  "message": "Access denied. Admin only."
```

```
}
```

Why?

The user is authenticated:

```
student + 1234 → Correct
```

But the user's role is:

```
user
```

The `/admin` route requires:

```
admin
```

Therefore:

```
Authentication → Successful
Authorization → Failed
```

---

16. Test 3 — Invalid Credentials

Create:

```
GET http://localhost:3000/admin
```

Headers:

Key	Value
`username`	`abc`
`password`	`1234`

Click:

```
Send
```

Expected Response

Status:

```
401 Unauthorized
```

Response:

```
{
  "message": "Authentication failed"
}
```

Why?

The username:

```
abc
```

does not exist in the users array.

Therefore:

```
Authentication → Failed
Authorization → Not performed
```

---

17. Test 4 — Normal User Route

The application also allows an authenticated user to access other routes.

For example:

```
GET http://localhost:3000/user
```

Headers:

Key	Value
`username`	`student`
`password`	`1234`

Expected response:

```
{
  "message": "Login successful",
  "user": "student",
  "role": "user"
}
```

---

18. HTTP Status Codes Used

200 — OK

The request was successful.

Used when:

- Authentication succeeds
- Authorization succeeds

---

401 – Unauthorized

Authentication failed.

Example:

```
{
  "message": "Authentication failed"
}
```

The username/password combination is incorrect.

---

403 – Forbidden

Authentication succeeded, but the user does not have permission.

Example:

```
{
  "message": "Access denied. Admin only."
}
```
